WOXSEN UNIVERSITY
SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR SCHOLARSHIP AND FINANCIAL AID MANAGEMENT

Program: B.Tech AIML

Section: Rhinos

Date of Submission: 14/10/2026

Academic Year: 2026–2027

Submitted by

Name: G. Uday Kiran Reddy

Roll Number: 25WU0102085

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing me with the opportunity to undertake this Project-Based Learning activity as part of the Database Management Systems course. I thank my course faculty for guidance, suggestions, and support throughout the project. The review process helped me connect database concepts—including ER modelling, relational schema design, normalization, SQL constraints, queries, and application development—with a practical institutional workflow. I also acknowledge the project review materials that guided the problem identification, logical database design, and application workflow.

UNDERTAKING

I hereby declare that the project entitled “Design and Implementation of a Database Management System for Scholarship and Financial Aid Management” is submitted as part of the Project-Based Learning activity for the Database Management Systems course. The report has been prepared for academic purposes using the project description and the three project review presentations as the primary source materials. Any implementation details not explicitly specified in those materials are identified as design assumptions or a proposed implementation baseline. External technical documentation is acknowledged in the references.

Signature: _____

Name: G. Uday Kiran Reddy

Roll Number: 25WU0102085

Program: [Enter Program]

Place: _____ Date: _____

TABLE OF CONTENTS

Contents	Page No.
Abstract	Update in Word after final pagination
List of Tables	Update in Word after final pagination
List of Figures	Update in Word after final pagination
List of Acronyms	Update in Word after final pagination
1. Problem Statement, Solution Strategy and ER Diagram	Update in Word after final pagination
1.1 Problem Statement	Update in Word after final pagination
1.2 Objectives	Update in Word after final pagination
1.3 Scope and Assumptions	Update in Word after final pagination
1.4 Proposed Solution Strategy	Update in Word after final pagination
1.5 Functional Requirements	Update in Word after final pagination
1.6 Entity-Relationship Diagram (ERD)	Update in Word after final pagination
1.7 Entity and Relationship Summary	Update in Word after final pagination
2. Logical Database Design	Update in Word after final pagination
2.1 Relational Schema	Update in Word after final pagination
2.2 Keys and Integrity Constraints	Update in Word after final pagination
2.3 Functional Dependencies	Update in Word after final pagination
2.4 Normalization up to 3NF	Update in Word after final pagination
2.5 Data Dictionary	Update in Word after final pagination
2.6 Implementation of Logical Design: DDL and Sample Data	Update in Word after final pagination
3. Application Development Using Logical Database Design	Update in Word after final pagination
3.1 Application Overview	Update in Word after final pagination
3.2 Development Environment and Database Connectivity	Update in Word after final pagination
3.3 Application Modules and Database Mapping	Update in Word after final pagination
3.4 CRUD Operations	Update in Word after final pagination
3.5 Business Rules, Validation and Error Handling	Update in Word after final pagination
3.6 Reports and Database Operations	Update in Word after final pagination
3.7 Application Screens	Update in Word after final pagination
3.8 Testing and Results	Update in Word after final pagination
4. Conclusion	Update in Word after final pagination
4.1 Limitations and Future Enhancements	Update in Word after final pagination
5. References	Update in Word after final pagination
6. Appendices	Update in Word after final pagination

ABSTRACT

Scholarship and financial aid administration involves scholarship scheme setup, eligibility assessment, application submission, document verification, approval, sanction, payment, and renewal. When these activities are maintained in disconnected spreadsheets or paper files, institutions can experience duplicate applications, inconsistent eligibility decisions, delayed disbursements, and weak traceability. This project designs a relational Database Management System (DBMS) to organize the scholarship lifecycle around a consistent and auditable data model. The design identifies students, schemes, eligibility criteria, applications, documents, verifications, approval stages, bank details, sanctions, disbursements, and renewals as the principal entities. The conceptual model is mapped to a relational schema and structured to satisfy First, Second, and Third Normal Forms. Primary keys, foreign keys, uniqueness rules, required fields, range checks, defaults, and indexes support data integrity and common retrieval tasks. The proposed application workflow covers scheme management, application processing, document verification, eligibility checking, approval, sanction, payment tracking, and reporting. SQL examples demonstrate joins, aggregate functions, nested eligibility checks, views, and operational reports. The resulting design provides a practical academic blueprint for reducing redundant records, improving workflow visibility, and supporting more reliable scholarship administration.

Keywords: Scholarship Management, Relational Database, ER Diagram, Third Normal Form, SQL, Financial Aid

CHAPTER 1
PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Scholarships and financial aid help students meet educational costs, but administering these programmes requires coordinated handling of student details, scholarship schemes, eligibility rules, supporting documents, reviewer decisions, sanctions, bank information, payments, and renewals. The Review-I presentation identifies fragmented spreadsheets and paper forms, manually applied eligibility checks, poor traceability, and slow disbursement as the main problems. Student, academic, and financial records may exist in separate files, making it difficult to maintain one consistent view of each application.

The absence of a centralized, rule-driven database can lead to duplicate applications, inconsistent decisions, delayed payments, and limited visibility for students, administrators, and auditors. The project therefore proposes a normalized relational database and a small database-connected application that records the lifecycle of an aid request and provides operational reports. The system is intended to improve consistency and traceability; it does not replace institutional policy, financial authorization, or bank-side payment processing.

1.2 Objectives

• Design a relational schema representing students, schemes, eligibility criteria, applications, documents, verifications, approvals, sanctions, bank details, disbursements, and renewals.

• Normalize the schema up to Third Normal Form (3NF) and define primary keys, foreign keys, uniqueness rules, and domain constraints.

• Implement database operations for scheme setup, application submission, document verification, eligibility assessment, approval, sanction, payment tracking, and renewal monitoring.

• Develop representative SQL queries, joins, aggregate reports, nested eligibility checks, and reusable views for operational decision-making.

• Validate core business rules and document positive and negative test cases for data integrity, duplicate prevention, and payment limits.

1.3 Scope and Assumptions

Table 1 defines the intended scope and the assumptions used to make the database design concrete. The Review presentations establish the lifecycle and major entities; the specific technology stack and some attribute-level details below are design choices for a reproducible academic prototype.

Item	Description
In Scope	Student and scheme records; eligibility criteria; applications by academic year; document submission and verification; approval stages; sanction recording; bank-detail validation; disbursement tracking; renewal status; SQL reporting and indexes.
Out of Scope / Exclusions	Actual transfer of funds through banking networks; integration with government portals or a production Student Information System; legal identity verification; production-grade authentication infrastructure; live document storage service.
Assumptions	One application per student, scheme, and academic year; one sanction record per application in the baseline design; partial payments may be recorded against a sanction; student bank details are maintained separately; cross-table rules are enforced by application transactions or database routines.
Implementation baseline	MySQL 8.0-compatible SQL and Python 3 with a simple GUI/database connector are proposed for implementation documentation. The review decks do not specify exact versions, IDE, connector, repository URL, or source-code files; these must be confirmed against the actual implementation before submission.

1.4 Proposed Solution Strategy

The solution uses a centralized relational database as the system of record. Users interact through role-specific workflows. Students submit applications and supporting documents; reviewers record verification results; authorized staff evaluate eligibility and approval stages; finance staff record sanctions and disbursements; administrators maintain schemes and generate reports. The application should use parameterized SQL, transactions for multi-step operations, and validation before a state transition is accepted.

Solution Element	Project-Specific Description

Primary Users / Roles	Student; document verifier/reviewer; department approver; financial-aid administrator; finance/payment operator; system administrator.
Major Application Modules	Scheme setup; student/application management; document verification; eligibility checking; approval stages; sanction management; bank details; payment tracking; renewal management; reporting.
Proposed Workflow	Create scheme and criteria → student applies for scheme/year → documents are submitted → mandatory documents are verified → eligibility is evaluated → approval stages are recorded → sanction is created → valid bank details are checked → one or more bounded disbursements are recorded → renewal is monitored.
Important Business Rules	Unique application per student/scheme/year; sanction must not exceed scheme limit; approval requires required verification; successful disbursements must not exceed sanctioned amount in total; active verified bank details are required before payment; renewal is unique per application/year.
Expected Reports / Outputs	Pending applications; eligibility results; document verification queue; approval status; sanction versus amount paid; disbursement history; renewals due; scheme-wise application and approval statistics.

1.5 Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	Create, view, update, and deactivate scholarship schemes and define eligibility criteria.	High
FR-02	Register student details and submit one application per scheme and academic year.	High
FR-03	Record document types and verification outcomes, including pending and rejected records.	High
FR-04	Evaluate eligibility and record sequential approval-stage decisions before sanction.	High
FR-05	Create a sanction within the scheme limit and record payment against verified bank details without exceeding the sanction total.	High
FR-06	Track renewal status and generate pending, approval, sanction, disbursement, renewal, and scheme-statistics reports.	High
FR-07	Validate input, prevent duplicate records, handle database errors, and preserve traceability of decisions.	High

Non-functional requirements include data privacy, role-based access, reliable transaction handling, understandable workflows, acceptable query performance, and an audit trail for

status changes. Credentials and sensitive bank details should not be exposed in logs or reports.

1.6 Entity-Relationship Diagram (ERD)

Figure 1 presents the proposed final ER model. It expands the preliminary Student–Scholarship–Application–Disbursement model from Review I into the full lifecycle entities shown in Review II. The entity names and key relationships are aligned with the relational schema in Chapter 2.

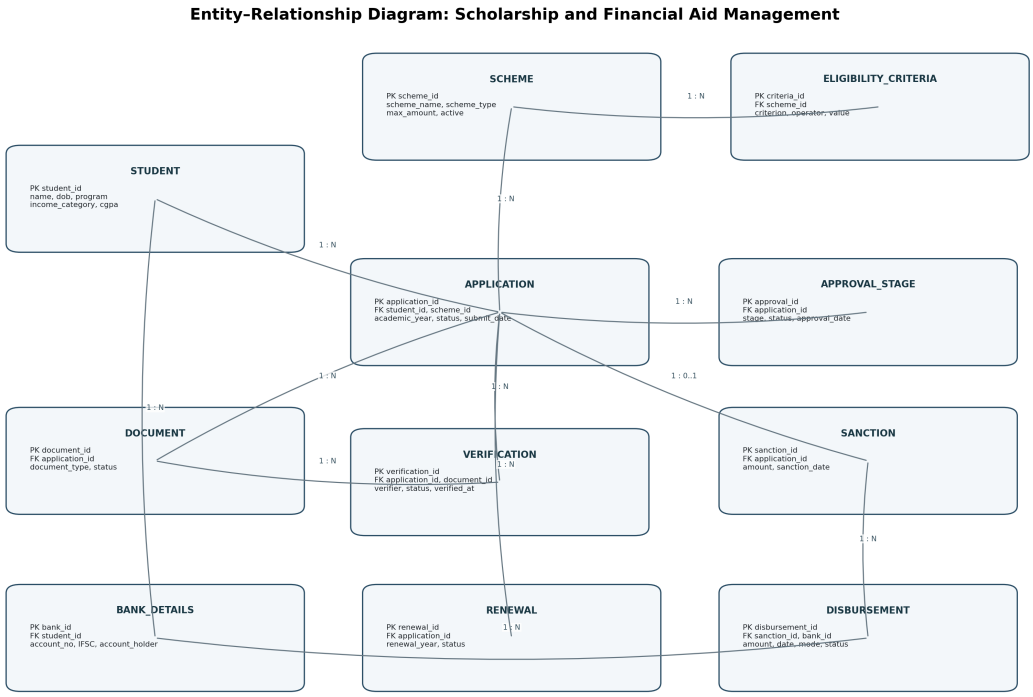


Figure 1. Entity-Relationship Diagram of the Proposed System

Cardinality summary: one student may submit many applications; one scheme may receive many applications and define many eligibility criteria; one application may have multiple documents, verification records, and approval stages; a student may maintain multiple bank-detail records; an application may have zero or one sanction in the baseline design; a sanction may be paid through multiple disbursements; and an application may have multiple renewal records across years.

1.7 Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
Student	student_id (PK), full_name, dob, program, income_category, annual_family_income, cgpa, email	Stores identity and academic/financial attributes; email is unique.
Scheme	scheme_id (PK), scheme_name, scheme_type, max_amount, eligibility thresholds, active	Stores award programme and upper funding limit.
Eligibility_Criteria	criteria_id (PK), scheme_id (FK), criterion_name, operator_code, criterion_value	Separates configurable criteria from scheme master data.
Application	application_id (PK), student_id (FK), scheme_id (FK), academic_year, status, submit_date	One application per student/scheme/year.
Document / Verification	document_id / verification_id (PK), application_id and document_id (FK), status, verifier, timestamp	Stores evidence and a traceable verification outcome.
Approval_Stage	approval_id (PK), application_id (FK), stage_name, status, reviewer, date	Records department, financial-aid, and final approval stages.
Bank_Details	bank_id (PK), student_id (FK), account_no, IFSC, verified, active	Separates payment destination from student profile.
Sanction / Disbursement	sanction_id / disbursement_id (PK), amount, date, bank_id, payment mode, status	Authorizes the award and records one or more payments within its limit.
Renewal	renewal_id (PK), application_id (FK), renewal_year, status, latest_cgpa	Tracks continuation eligibility and decisions across academic years.

CHAPTER 2
LOGICAL DATABASE DESIGN

This chapter maps the ER model into relations and documents key constraints, functional dependencies, normalization, data types, and representative SQL. The Review-II deck states that the schema was normalized to 3NF, the data dictionary was completed, and a prototype database, sample data, queries, and views were prepared.

2.1 Relational Schema

S.No.	Table Name	Primary Key	Important Foreign Keys / Purpose
1	Student	student_id	Student profile and eligibility attributes.
2	Scheme	scheme_id	Scheme configuration, type, award cap and base thresholds.
3	Eligibility_Criteria	criteria_id	scheme_id → Scheme; one row per scheme criterion.
4	Application	application_id	student_id → Student; scheme_id → Scheme; unique student/scheme/year.
5	Document	document_id	application_id → Application; submitted evidence.
6	Verification	verification_id	application_id → Application; document_id → Document; verification outcome.
7	Approval_Stage	approval_id	application_id → Application; one row per application/stage.
8	Bank_Details	bank_id	student_id → Student; payment destination details.
9	Sanction	sanction_id	application_id → Application; one sanction per application in baseline.
10	Disbursement	disbursement_id	sanction_id → Sanction; bank_id → Bank_Details; payment transaction.
11	Renewal	renewal_id	application_id → Application; unique application/renewal

2.2 Keys and Integrity Constraints

Primary keys provide stable identifiers. Foreign keys maintain referential integrity, and UNIQUE constraints prevent duplicate applications, duplicate scheme names, repeated approval stages, duplicate bank-account/IFSC pairs, and repeated renewal years. NOT NULL constraints identify mandatory facts; CHECK constraints validate amounts and ranges;

DEFAULT values provide safe initial statuses. The SQL appendix contains representative DDL.

Table / Area	Key / Constraint	Attribute(s)	Business Rule Enforced
Student	PK / UNIQUE / NOT NULL	student_id; email; required profile fields	Each student is uniquely identified; email cannot be duplicated.
Scheme	PK / UNIQUE / CHECK	scheme_id; scheme_name; max_amount > 0; min_cgpa 0–10	Scheme name is unique; award amount and CGPA range are valid.
Application	FK / UNIQUE / DEFAULT	student_id; scheme_id; UNIQUE(student_id, scheme_id, academic_year)	References valid student and scheme; prevents duplicate application per year.
Document	FK / NOT NULL / DEFAULT	application_id; document_type; status	Every document belongs to an application and begins with a defined status.
Verification	FK / UNIQUE	application_id; document_id; status	Verification refers to existing records; baseline allows one verification record per document.
Approval_Stage	FK / UNIQUE	application_id; stage_name; status	Prevents duplicate records for the same application stage.
Bank_Details	FK / UNIQUE / CHECK	student_id; account_no + IFSC; IFSC format	Bank account is linked to a student; duplicate account/IFSC pair is blocked.
Sanction	FK / UNIQUE / CHECK	application_id; sanctioned_amount > 0	One sanction per application in baseline; amount must be positive.
Disbursement	FK / UNIQUE / CHECK	sanction_id; bank_id; amount > 0; transaction_reference	Payment refers to a sanction and bank record; transaction reference is unique.
Renewal	FK / UNIQUE / CHECK	application_id; renewal_year; latest_cgpa 0–10	Renewal is unique per application/year and CGPA is within range.

Cross-table constraints need more than a simple row-level CHECK in MySQL. Before sanction, the application service or stored procedure must confirm that required documents are verified and approval stages are complete, and that the sanction does not exceed the scheme maximum. Before payment, a transaction must confirm that the bank record is active and verified and that the sum of successful disbursements plus the new payment does not exceed the sanction. These checks should be performed under an appropriate transaction/ locking strategy to prevent concurrent overpayment.

2.3 Functional Dependencies

The principal functional dependencies used in normalization are:

• student_id → full_name, dob, program, income_category, annual_family_income, cgpa, email, phone

• scheme_id → scheme_name, scheme_type, max_amount, min_cgpa, max_family_income, eligible_program, active

• criteria_id → scheme_id, criterion_name, operator_code, criterion_value, mandatory

• application_id → student_id, scheme_id, academic_year, status, submit_date, remarks

• (student_id, scheme_id, academic_year) → application_id, status, submit_date, remarks

• document_id → application_id, document_type, reference_uri, status, submitted_at

• verification_id → application_id, document_id, verifier_id, status, comments, verified_at

• approval_id → application_id, stage_name, status, reviewer_id, approval_date, comments

• bank_id → student_id, account_holder, account_no, ifsc, bank_name, verified, active

• sanction_id → application_id, sanctioned_amount, sanction_date, approved_by, status

• disbursement_id → sanction_id, bank_id, amount, disbursement_date, payment_mode, transaction_reference, status

• renewal_id → application_id, renewal_year, status, latest_cgpa, submitted_at, decision_date, remarks

2.4 Normalization up to Third Normal Form (3NF)

The design separates independently changing facts into related tables rather than storing student, scheme, document, approval, and payment details repeatedly in one large application table. Each table uses atomic attributes, and repeating groups such as multiple documents or payments are represented as separate rows. This avoids update, insertion, and deletion anomalies.

Relation Name	1NF	2NF	3NF	Remarks / Decomposition
Student	Yes	Yes	Yes	Single-valued profile attributes depend on student_id.
Scheme	Yes	Yes	Yes	Scheme attributes depend on scheme_id; criteria are separated.

Eligibility_Criteria	Yes	Yes	Yes	Each criterion is a separate row; no repeating criterion columns.
Application	Yes	Yes	Yes	Application facts depend on application_id; alternate unique key enforces business identity.
Document / Verification	Yes	Yes	Yes	Evidence and verification facts are separated from the application.
Approval_Stage	Yes	Yes	Yes	Approval decisions are stored per stage, not as repeating columns.
Bank_Details	Yes	Yes	Yes	Bank attributes depend on bank_id and are not copied into each payment.
Sanction / Disbursement	Yes	Yes	Yes	Authorized amount is separated from individual payment transactions.
Renewal	Yes	Yes	Yes	Each renewal-year decision is a separate record.

1NF is satisfied by atomic values and the absence of repeating groups. 2NF is satisfied because non-key attributes depend on the whole candidate key; most relations use a single-column surrogate key, while declared composite uniqueness rules represent alternate business keys. 3NF is supported by separating entities whose attributes depend on their own identifiers—for example, scheme details are not repeated in Application, student bank details are not embedded in Disbursement, and verification details are not stored as repeated columns on Application. The schema is intended to be in 3NF; formal BCNF analysis is not claimed for every possible business extension.

2.5 Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
Student	student_id	VARCHAR(15)	PK, NOT NULL	Institutional student identifier.
Student	full_name	VARCHAR(100)	NOT NULL	Student’s name.
Student	dob	DATE	NOT NULL	Date of birth.
Student	program	VARCHAR(80)	NOT NULL, indexed with CGPA	Academic programme.
Student	income_category	ENUM	NOT NULL	Categorical family-income band.
Student	annual_family_income	DECIMAL(12,2)	CHECK >= 0	Annual family income.
Student	cgpa	DECIMAL(3,2)	CHECK 0–10	Cumulative grade point average.
Scheme	scheme_id	INT	PK, AUTO_INCREMENT	Scheme identifier.
Scheme	scheme_name	VARCHAR(120)	UNIQUE, NOT NULL	Display name of scholarship scheme.
Scheme	scheme_type	ENUM	NOT NULL	Merit, need-based, research, or donor-funded.
Scheme	max_amount	DECIMAL(12,2)	CHECK > 0	Maximum award amount.
Eligibility_Criteria	criteria_id	INT	PK, AUTO_INCREMENT	Criterion identifier.
Eligibility_Criteria	scheme_id	INT	FK → Scheme	Parent scheme.
Application	application_id	INT	PK, AUTO_INCREMENT	Application identifier.
Application	student_id	VARCHAR(15)	FK → Student	Applicant.
Application	scheme_id	INT	FK → Scheme	Requested scheme.
Application	academic_year	VARCHAR(9)	UNIQUE with student/ scheme	Academic year of application.
Application	status	ENUM	NOT NULL, DEFAULT	Current application state.
Document	document_id	INT	PK, AUTO_INCREMENT	Document identifier.
Document	application_id	INT	FK → Application	Application supported by the document.
Document	document_type	ENUM	NOT NULL	Type of submitted evidence.
Verification	verification_id	INT	PK, AUTO_INCREMENT	Verification record identifier.
Verification	document_id	INT	FK, UNIQUE	Document being verified.
Approval_Stage	approval_id	INT	PK, AUTO_INCREMENT	Approval stage record identifier.
Approval_Stage	stage_name	ENUM	UNIQUE per application	Department, aid office, or final authority.
Bank_Details	bank_id	INT	PK, AUTO_INCREMENT	Bank record identifier.
Bank_Details	account_no	VARCHAR(34)	UNIQUE with IFSC	Account number; sensitive data.
Bank_Details	ifsc	VARCHAR(11)	CHECK pattern	Indian bank branch code.
Sanction	sanction_id	INT	PK, AUTO_INCREMENT	Sanction identifier.
Sanction	sanctioned_amount	DECIMAL(12,2)	CHECK > 0	Authorized award amount.
Disbursement	disbursement_id	INT	PK, AUTO_INCREMENT	Payment record identifier.
Disbursement	amount	DECIMAL(12,2)	CHECK > 0	Amount of one payment transaction.
Disbursement	transaction_reference	VARCHAR(100)	UNIQUE, NOT NULL	External/internal payment reference.
Renewal	renewal_id	INT	PK, AUTO_INCREMENT	Renewal record identifier.
Renewal	renewal_year	VARCHAR(9)	UNIQUE per application	Academic year under renewal review.

2.6 Implementation of Logical Design: DDL and Sample Data

The SQL implementation baseline in Appendix A targets MySQL 8.0 or later and includes table definitions, integrity constraints, indexes, illustrative sample records, reporting queries, and a view. The review materials provide illustrative applications A-1042 through A-1045 and award amounts of ₹45,000, ₹30,000, and ₹80,000. The appendix uses comparable illustrative records while avoiding a claim that these SQL statements were executed in the supplied environment.

Item	Details
DBMS	MySQL 8.0+ (proposed baseline; confirm installed version)
Number of Tables	11 core relations
Sample Data	Illustrative students, three schemes, applications, documents, verifications, approval stages, bank details, sanctions, disbursements, and renewals
Indexes	Application status/year; application scheme; student program/CGPA; disbursement date/status
Script	Appendix_A_Scholarship_DBMS.sql

Representative DDL:

```
CREATE TABLE Application (
  application_id INT AUTO_INCREMENT PRIMARY KEY,
  student_id VARCHAR(15) NOT NULL,
  scheme_id INT NOT NULL,
  academic_year VARCHAR(9) NOT NULL,
  status ENUM('SUBMITTED','PENDING_DOCUMENTS','UNDER_REVIEW',
              'VERIFIED','APPROVED','REJECTED','SANCTIONED',
              'PARTIALLY_DISBURSED','DISBURSED','RENEWAL_DUE')
              NOT NULL DEFAULT 'SUBMITTED',
  submit_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (student_id, scheme_id, academic_year),
  FOREIGN KEY (student_id) REFERENCES Student(student_id),
  FOREIGN KEY (scheme_id) REFERENCES Scheme(scheme_id)
);
```

CHAPTER 3
APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The application layer provides a guided workflow over the relational database rather than maintaining a separate collection of static records. The Review-III presentation identifies eight modules: scheme management, student application, document verification, eligibility checking, approval, sanction, payment tracking, and reporting. The UI should expose only the operations appropriate to each role and should refresh displayed status from committed database records.

3.2 Development Environment and Database Connectivity

The supplied review decks do not name the exact programming language, GUI framework, connector, operating system, repository, or version numbers. To make the report reproducible, the following table describes a proposed implementation baseline; replace these details with the actual project configuration before final submission.

Item	Details
Programming Language	Python 3 (proposed baseline; Java is also permitted by the report template)
DBMS	MySQL 8.0+
IDE / GUI Toolkit	VS Code or PyCharm; Tkinter desktop GUI (proposed)
Database Connector / Driver	mysql-connector-python (proposed)
Operating Environment	Windows/Linux desktop with MySQL server
Repository Link	Not provided in the supplied review files; insert the actual repository URL if available

Database connection should be configured outside source code using environment variables or a protected configuration file. The application should use parameterized statements rather than concatenating user input into SQL. Each multi-step decision should run within a transaction; commit follows successful validation and writes, while any exception triggers rollback and a user-safe error message.

3.3 Application Modules and Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Scheme Management	Create, edit, activate/deactivate schemes and define criteria.	Scheme, Eligibility_Criteria	Create/read/update; validate award cap and thresholds.
Student & Application	Capture applicant details and submit scheme/year application.	Student, Application	Create/read/update; unique application check.
Document Verification	Review submitted evidence and record outcomes.	Document, Verification	Read/update; pending/verified/ rejected status.
Eligibility Checking	Compare student attributes with scheme criteria.	Student, Scheme, Eligibility_Criteria, Application	Read; rule evaluation and eligibility result.
Approval	Record reviewer decisions at each approval stage.	Approval_Stage, Application, Verification	Create/read/update; block approval until verification is complete.
Sanction	Record authorized amount after approval.	Sanction, Scheme, Application, Approval_Stage	Create/read; enforce scheme maximum.
Payment Tracking	Validate bank details and record disbursements.	Bank_Details, Sanction, Disbursement	Read/create; transaction, cumulative payment limit.
Renewal & Reports	Track renewal eligibility and provide operational summaries.	Renewal plus relevant lifecycle tables	Create/read/update; joins, aggregates, views.

3.4 CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence / Expected Behaviour
Create	Add scheme; submit application; upload document metadata; create approval/ sanction/payment/renewal record.	Scheme, Eligibility_Criteria, Application, Document, Approval_Stage, Sanction, Disbursement, Renewal	New record is committed only after required validation.
Read / Search	Search student, scheme, application status, verification queue, payment history.	Student, Scheme, Application, Document, Verification, Disbursement	Parameterized filters return matching database rows.
Update	Change scheme settings, verification outcome, approval status, bank verification, renewal state.	Scheme, Document, Verification, Approval_Stage, Bank_Details, Renewal	Update is authorized and preserves valid status transitions.
Delete / Controlled Removal	Deactivate a scheme or correct test records; avoid deleting financial history.	Scheme, Application, Disbursement and related tables	Prefer soft deactivation; restrictive foreign keys protect financial records.

3.5 Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Duplicate application prevention	Database UNIQUE(student_id, scheme_id, academic_year) plus friendly pre-check.	“An application already exists for this scheme and academic year.”
Eligibility and verification before approval	Check mandatory criteria and required document verification before allowing approval transition.	“Approval is unavailable until required documents are verified.”
Sanction within scheme limit	Compare requested sanction with Scheme.max_amount inside a transaction.	“Sanction amount exceeds the scheme maximum.”
Disbursement within sanction	Lock/read sanction totals transactionally; ensure successful paid total plus new amount is within sanction.	“Payment exceeds the remaining sanctioned balance.”
Valid bank details	Require active and verified Bank_Details record before payment.	“Select an active, verified bank account.”
Input and database errors	Validate dates, numeric ranges, mandatory fields; use parameterized SQL and exception handling; rollback failed transactions.	Display actionable validation messages without exposing SQL credentials or stack traces.

3.6 Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Pending applications	JOIN across Application, Student and Scheme; WHERE status filter; view vw_pending_applications.	Application ID, student, scheme, academic year, status.
Eligible students	JOIN/CROSS JOIN with threshold filters; nested predicates or rule-engine evaluation.	Students meeting a scheme's base eligibility criteria.
Approval status	JOIN Application with Approval_Stage and Verification.	Current stage, reviewer decision, date, unresolved checks.
Sanction and disbursement summary	LEFT JOIN, SUM, COALESCE, GROUP BY.	Sanctioned amount, amount paid, remaining balance.
Renewals due	JOIN Renewal with Application, Student and Scheme; WHERE renewal status = DUE.	Student, scheme, renewal year, next action.
Scheme statistics	COUNT, conditional aggregation, GROUP BY.	Application counts and approved/rejected totals per scheme.

3.7 Application Screens

The Review-III deck describes the screen functions but the supplied PDF does not contain actual application screenshots. The following are therefore labelled as screen specifications, not screenshots of a verified running application. Replace them with cropped screenshots from the final application before submission.

Fields: scheme name, type, maximum amount, minimum CGPA, maximum family income, eligible programme and active flag. Actions: Create, Search, Update, Save and Define Criteria.

Figure 2. Scheme Management Screen — proposed layout

Application search results with student/scheme/year/status; document list with type, submission reference and verification status; actions to verify, reject, or request a replacement.

Figure 3. Application and Document Verification Screen — proposed layout

Approval-stage history, scheme cap, sanction amount, bank verification status, payment amount/date/mode/reference and remaining balance. Payment action is disabled when bank details are unverified or the amount exceeds the balance.

Figure 4. Approval, Sanction and Payment Screen — proposed layout

Filters for academic year and scheme; report panels for pending applications, eligible students, approvals, sanctions, disbursements, renewals due and scheme statistics.

Figure 5. Reports Dashboard — proposed layout

3.8 Testing and Results

The Review-III presentation states that the application was implemented and tested and lists validation targets such as invalid input, duplicate applications, missing documents, invalid bank details, and invalid amounts. However, no executable source code, test log, or actual application screenshots were included with the supplied files. Accordingly, the matrix below is a test plan aligned with the review evidence; the actual-result and status fields must be confirmed by executing the application and database before submission.

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Scheme setup	Create scheme with positive amount and valid CGPA threshold.	Scheme and criteria saved; scheme appears in search.	Not evidenced in supplied files; execute test.	To verify
TC-02	Duplicate application	Submit same student/ scheme/year twice.	Second insert rejected by UNIQUE constraint and friendly message.	Not evidenced in supplied files; execute test.	To verify
TC-03	Document verification	Attempt approval while a required document is pending.	Approval transition blocked with explanation.	Not evidenced in supplied files; execute test.	To verify
TC-04	Eligibility checking	Submit student with CGPA below scheme minimum.	Eligibility result is not eligible; approval cannot proceed.	Not evidenced in supplied files; execute test.	To verify

TC-05	Sanction validation	Enter sanction greater than scheme maximum.	Transaction rejected; no sanction is committed.	Not evidenced in supplied files; execute test.	To verify
TC-06	Payment validation	Attempt payment with unverified bank or amount above remaining sanction.	Payment rejected and transaction rolled back.	Not evidenced in supplied files; execute test.	To verify
TC-07	Reporting	Run pending application and scheme-statistics reports.	Report values match rows stored in database.	Not evidenced in supplied files; execute test.	To verify

Expected results demonstrate how the design should behave, but they are not substitutes for observed results. For final assessment, run each case against the implemented database, capture the output, record actual results, and change Status to Pass or Fail based on evidence. A separate concurrency test should attempt two payments against the same remaining balance to confirm that transaction handling prevents overpayment.

CHAPTER 4
CONCLUSION

This project develops a structured database design for scholarship and financial aid management, addressing the fragmented data, manual eligibility evaluation, delayed processing, and limited traceability identified in Review I. The proposed ER model covers the lifecycle from student and scheme setup through application, documents, verification, approvals, sanction, bank details, disbursement, and renewal. Its relational mapping separates independent facts, uses keys and integrity constraints, and is designed to satisfy 3NF. Representative SQL demonstrates operational joins, aggregation, eligibility filtering, views, and reports.

The application workflow defined in Review III maps the database to practical modules for scheme management, application processing, document verification, eligibility checking, approval, sanction, payment tracking, and reporting. The SQL appendix provides a MySQL-oriented implementation baseline and sample data for academic use. Before final submission, the proposed environment details, actual execution results, screenshots, repository reference, and faculty/student metadata should be updated from the running project so the report accurately distinguishes completed implementation evidence from design documentation.

4.1 Limitations and Future Enhancements

• Production authentication, fine-grained role permissions, encrypted secrets, and audit-log retention require further implementation and security review.

• Eligibility criteria are represented as configurable records, but a generalized rule engine for arbitrary operators and complex combinations would need additional logic and validation.

• Bank transfer execution and reconciliation with payment gateways are outside the prototype; future work could integrate an approved payment provider and reconciliation workflow.

• Actual UI screenshots, a repository link, observed test logs, and the installed DBMS/ connector versions must be added after validating the running implementation.

• Future enhancements may include email/SMS notifications, document upload storage, dashboard visualizations, scholarship budget allocation, and exportable audit reports.

CHAPTER 5
REFERENCES

[1] G. Uday Kiran Reddy, “Design and Implementation of a Database Management System for Scholarship and Financial Aid Management System,” DBMS Project Review 1 presentation, Woxsen University, 2026.

[2] “Design & Implementation of a Database Management System: Scholarship & Financial Aid Management,” DBMS Review 2 presentation, Oct. 13, 2026.

[3] “Scholarship & Financial Aid Management System: Application Implementation, Testing & Final Demonstration,” DBMS Review 3 presentation, 2026.

[4] Oracle, “MySQL 8.0 Reference Manual,” [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/. Accessed: Oct. 15, 2026.

[5] Oracle, “MySQL Connector/Python Developer Guide,” [Online]. Available: https://dev.mysql.com/doc/connector-python/en/. Accessed: Oct. 15, 2026.

[6] Python Software Foundation, “Python 3 Documentation,” [Online]. Available: https://docs.python.org/3/. Accessed: Oct. 15, 2026.

CHAPTER 6
APPENDICES

APPENDIX A — COMPLETE DATABASE SCRIPT

The complete SQL script is supplied as a separate companion file: Appendix_A_Scholarship_DBMS.sql. It includes database/table creation, constraints, indexes, sample records, reporting queries, and a reusable view. Execute it only in a test database after reviewing the sample data and local MySQL version.

APPENDIX B — APPLICATION SOURCE CODE

The supplied review files do not include a source-code archive or repository URL. Insert the actual source-code folder or repository reference here. Do not state a repository URL unless it exists and is accessible.

APPENDIX C — ADDITIONAL TEST DATA AND OUTPUTS

Attach actual screenshots for scheme setup, application submission, document verification, eligibility results, approval/sanction, payment tracking, and at least one report. Add the observed test log and any additional sample outputs after running the application.

Submission checklist: replace all bracketed personal/course placeholders; confirm DBMS and programming environment; execute Appendix A and correct any environment-specific issues; record observed test results; insert actual application screenshots; update page numbers and automatic tables in Word.